EXP1

Exploring Matplotlib, Seaborn and Plotly Visualization Tools

AIM:

To explore the Matplotlib, Seaborn and Plotly visualization libraries by loading the Titanic dataset, inspecting and cleaning it, and drawing a bar chart, pie chart, histogram, box plot and scatter plot.

ALGORITHM:

1. Import the required libraries: pandas, numpy, matplotlib and seaborn.
2. Load the Titanic dataset (Titanic-Dataset.csv) into a DataFrame using read_csv().
3. Display the first five and last five rows using head() and tail().
4. Check the size of the data using shape, and its structure using info() and describe().
5. Find the number of missing values in each column using isnull().sum().
6. Fill the missing Age values with the median, fill the missing Embarked values with the mode, and drop the Cabin column.
7. Check the missing values again to confirm that the data is clean.
8. Draw a bar chart of the number of passengers in each class using Plotly Express.
9. Draw a pie chart of survived and not survived passengers using the pandas plot (Matplotlib) function.
10. Draw a histogram with a density curve of passenger age using Seaborn.
11. Draw a box plot of passenger age using Seaborn.
12. Draw a scatter plot of Age against Fare, coloured by Survived, using Seaborn.
13. Observe the charts and note the results.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv("Titanic-Dataset.csv")
```

```
print(df.head())
```

```
print(df.tail())
```

```
print("Shape:", df.shape)
```

```
df.info()
```

```
df.describe()
```

```
print(df.isnull().sum())
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())

# Fill missing Embarked values with the mode
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Drop Cabin column
df.drop(columns=["Cabin"], inplace=True)
```

```
print(df.isnull().sum())
```

```
import plotly.express as px

fig = px.bar(
    x=df["Pclass"].value_counts().sort_index().index,
    y=df["Pclass"].value_counts().sort_index().values,
    labels={
        "x": "Passenger Class",
        "y": "Number of Passengers"
    },
    title="Number of Passengers by Class"
)

fig.show()
```

```
df["Survived"].value_counts().plot(
    kind="pie",
    figsize=(6,6),
    labels=["Not Survived", "Survived"],
    autopct="%1.1f%%",
    startangle=90,
    shadow=True
)

plt.title("Passenger Survival Distribution")
plt.ylabel("")

plt.show()
```

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

sns.histplot(
    data=df,
    x="Age",
    bins=20,
    kde=True
)

plt.title("Distribution of Passenger Age")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,5))

sns.boxplot(data=df, y="Age")

plt.title("Box Plot of Passenger Age")
plt.ylabel("Age")

plt.show()
```

```
plt.figure(figsize=(8,5))

sns.scatterplot(
    data=df,
    x="Age",
    y="Fare",
    hue="Survived"
)

plt.title("Age vs Fare by Survival")
plt.xlabel("Age")
plt.ylabel("Fare")

plt.show()
```

OUTPUT:

Output of: print(df.head())

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

Output of: print(df.tail())

```
     PassengerId  Survived  Pclass                                      Name  \
886          887         0       2                     Montvila, Rev. Juozas
887          888         1       1              Graham, Miss. Margaret Edith
888          889         0       3  Johnston, Miss. Catherine Helen "Carrie"
```

```
889          890         1       1                     Behr, Mr. Karl Howell
890          891         0       3                       Dooley, Mr. Patrick

        Sex   Age  SibSp  Parch      Ticket   Fare Cabin Embarked
886    male  27.0      0      0      211536  13.00   NaN        S
887  female  19.0      0      0      112053  30.00   B42        S
888  female   NaN      1      2  W./C. 6607  23.45   NaN        S
889    male  26.0      0      0      111369  30.00  C148        C
890    male  32.0      0      0      370376   7.75   NaN        Q
```

Output of: print("Shape:", df.shape)

```
Shape: (891, 12)
```

Output of: df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 118.9 KB
```

Output of: df.describe()

```
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

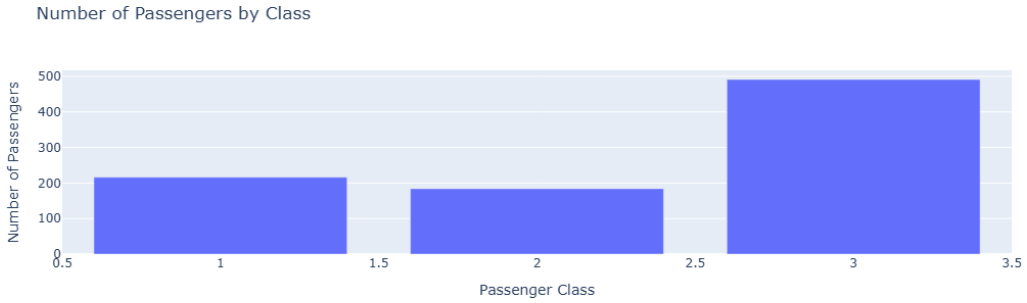
Output of: print(df.isnull().sum())

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

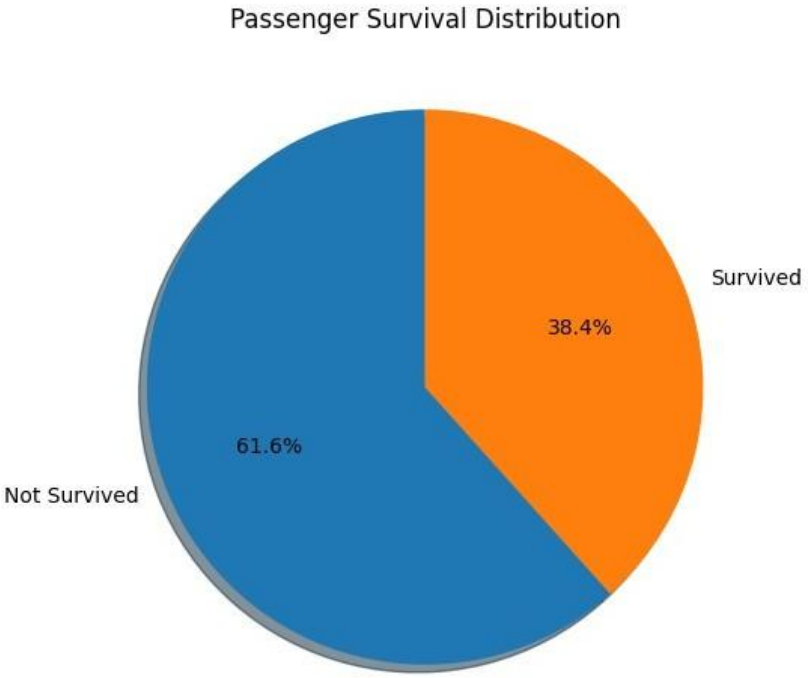
Output of: print(df.isnull().sum())

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

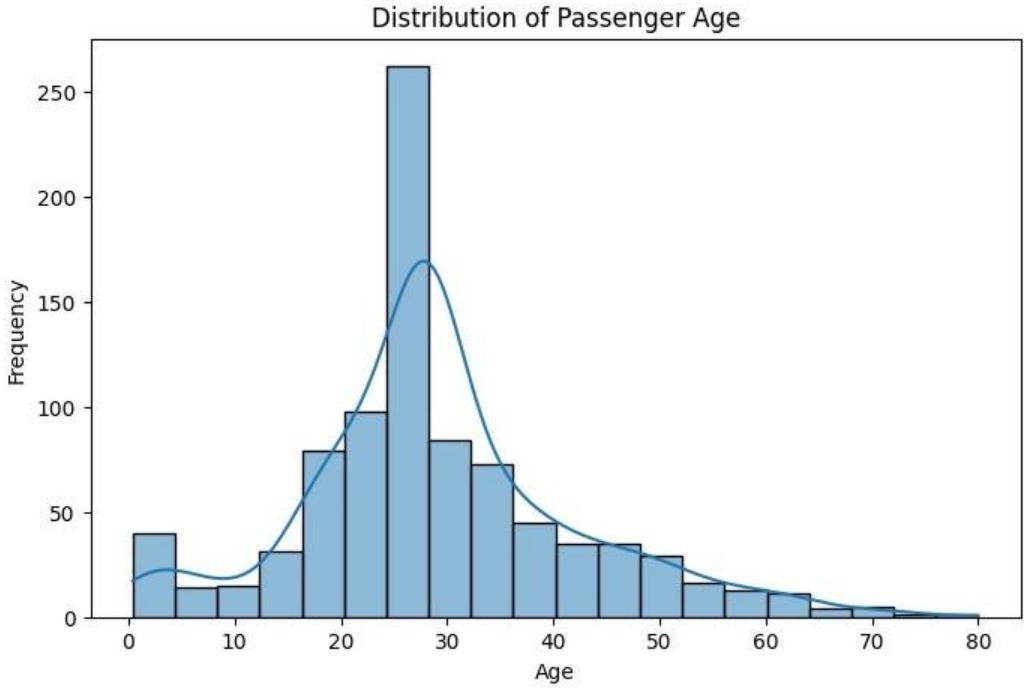
Output of: import plotly.express as px



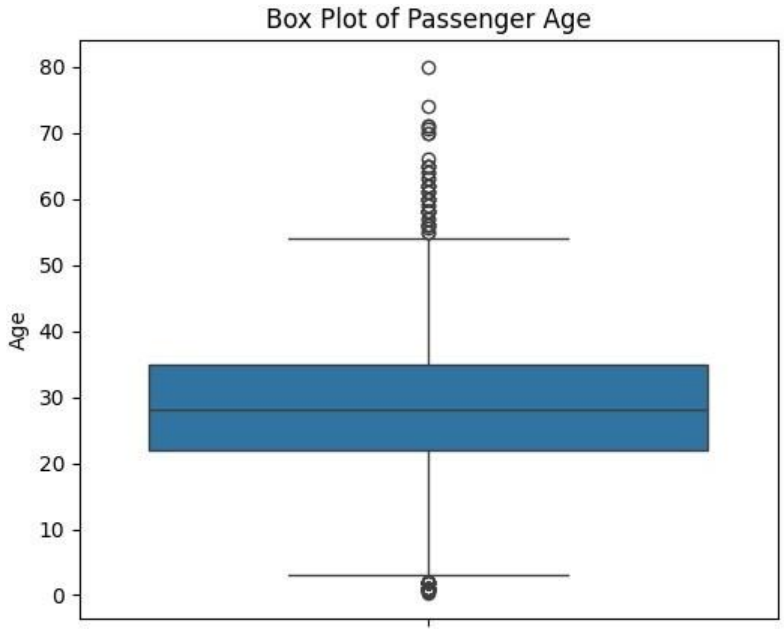
Output of: df["Survived"].value_counts().plot(



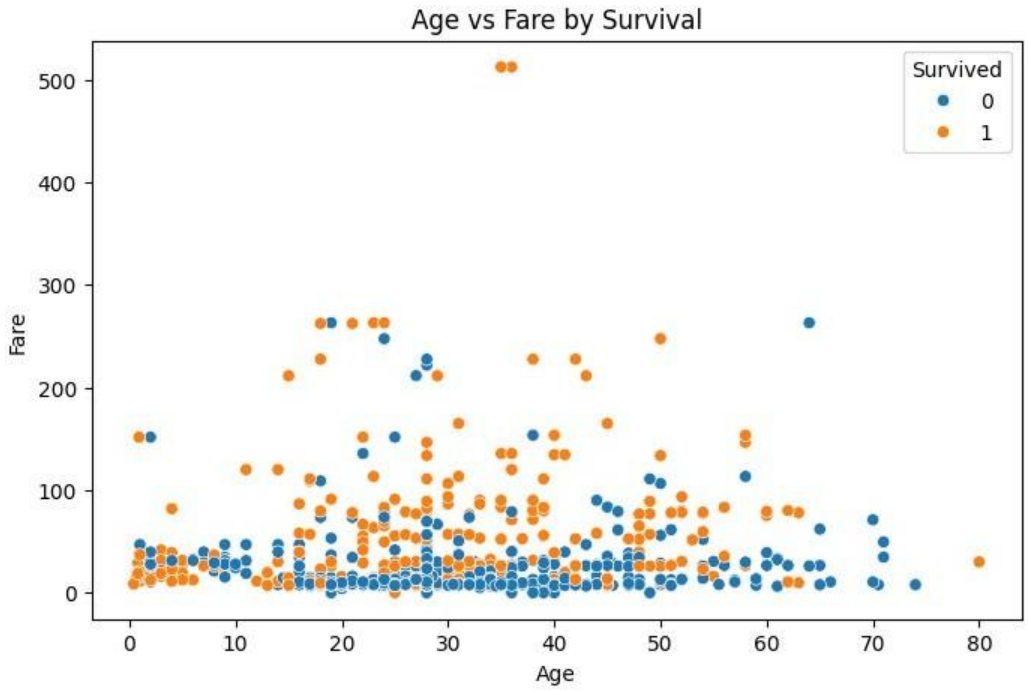
Output of: import seaborn as sns



Output of: import seaborn as sns



Output of: plt.figure(figsize=(8,5))



RESULT:

Thus, the Matplotlib, Seaborn and Plotly libraries were explored using the Titanic dataset (891 rows and 12 columns). The missing values in Age (177), Embarked (2) and Cabin (687) were handled, and the bar chart, pie chart, histogram, box plot and scatter plot were drawn successfully.

EXP 2

IPYWidgets-Based Shoping Cart and billing System

AIM:

To set up the Python environment by installing and checking the ipywidgets and jupyterlab_widgets libraries, and to use Jupyter widgets to build an interactive order form with input fields, buttons and tabs that calculates a bill.

ALGORITHM:

1. Import ipywidgets and the display and clear_output functions from IPython.display.
2. Create the text input widgets for Name, Email and Address.
3. Create the Product dropdown and the Quantity, Discount and Tax sliders.
4. Create the Gift Wrap checkbox, the Colour picker and the Payment toggle buttons (Cash, UPI, Card).
5. Create the Add to Cart, Checkout and Clear buttons and an Output widget.
6. Store the product prices in a dictionary and create an empty list for the cart.
7. Define add_item() to add the selected product and quantity to the cart and show the details.
8. Define bill() to check that the cart is not empty, calculate the total, discount, tax and final amount, and print the bill.
9. Define clear_cart() to empty the cart and show a message.
10. Connect each button to its function using on_click().
11. Arrange the widgets in three tabs (Customer Details, Product Selection, Order Summary) and display them.
12. Install or upgrade jupyterlab_widgets and ipywidgets using %pip so that the widgets are displayed correctly.

CODE:

```
import ipywidgets as widgets
from IPython.display import display, clear_output
```

```
name = widgets.Text(description="Name:")
email = widgets.Text(description="Email:")
address = widgets.Textarea(description="Address:")
```

```
product = widgets.Dropdown(
    options=["Laptop", "Mobile", "Headphones"],
    description="Product:"
)
quantity = widgets.IntSlider(min=1, max=10, value=1, description="Qty:")
discount = widgets.IntSlider(min=0, max=30, step=5, description="Discount:")
tax = widgets.FloatSlider(min=0, max=30, value=18, description="Tax:")
```

```
gift = widgets.Checkbox(description="Gift Wrap")
color = widgets.ColorPicker(description="Color:")
payment = widgets.ToggleButtons(
    options=["Cash", "UPI", "Card"],
```

```
    description="Payment:"
)
```

```
add = widgets.Button(description="Add to Cart", button_style="success")
checkout = widgets.Button(description="Checkout", button_style="primary")
clear = widgets.Button(description="Clear", button_style="danger")
output = widgets.Output()
```

```
prices = {
    "Laptop": 50000,
    "Mobile": 25000,
    "Headphones": 2000
}
cart = []
```

```
def add_item(b):
    cart.append((product.value, quantity.value))
    with output:
        clear_output()
        print("Added to Cart!")
        print("---------------------------------")
        print("Product:", product.value)
        print("Quantity:", quantity.value)
        print("Price: $", prices[product.value])


# Checkout
def bill(b):
    with output:
        clear_output()

        if not cart:
            print("Cart is empty!")
            return

        total = sum(prices[p] * q for p, q in cart)

        discount_amount = total * discount.value / 100
        after_discount = total - discount_amount

        tax_amount = after_discount * tax.value / 100
        final = after_discount + tax_amount

        print("Customer:", name.value)
        print("Email:", email.value)
        print("Address:", address.value)
        print("---------------------------------")

        for p, q in cart:
            print(p, "x", q)

        print("---------------------------------")
        print("Total: $", total)
        print("Discount:", discount.value, "%")
        print("Tax:", tax.value, "%")
        print("Final Amount: $", final)
        print("Payment:", payment.value)
        print("Gift Wrap:", gift.value)
```

```
        print("Order Confirmed!")
def clear_cart(b):
    cart.clear()
    with output:
        clear_output()
        print("Cart Cleared!")


# Button Events
add.on_click(add_item)
checkout.on_click(bill)
clear.on_click(clear_cart)
```

```
tabs = widgets.Tab([
    widgets.VBox([
        widgets.HBox([name, email]),
        address,
    ]),

    widgets.VBox([
        product,
        quantity,
        discount,
        tax,
        payment,
        gift,
        color,
        widgets.HBox([add, clear])
    ]),

    widgets.VBox([
        checkout,
        output
    ])
])

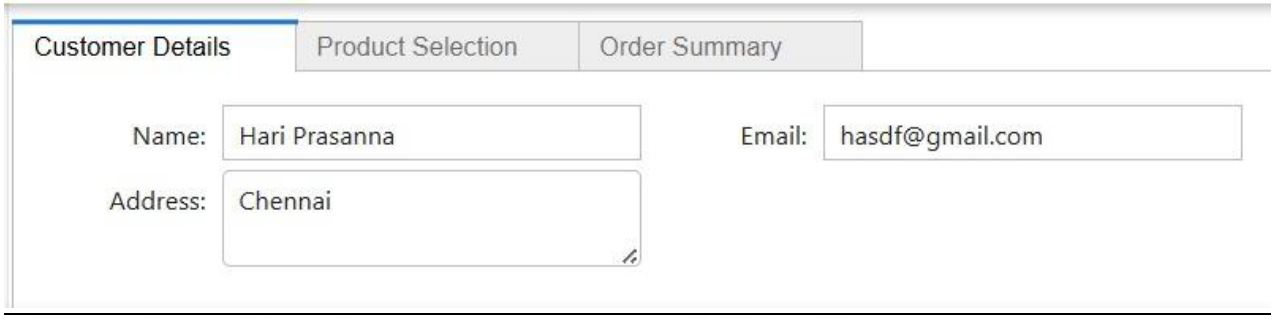
# Tab Names
tabs.set_title(0, "Customer Details")
tabs.set_title(1, "Product Selection")
tabs.set_title(2, "Order Summary")

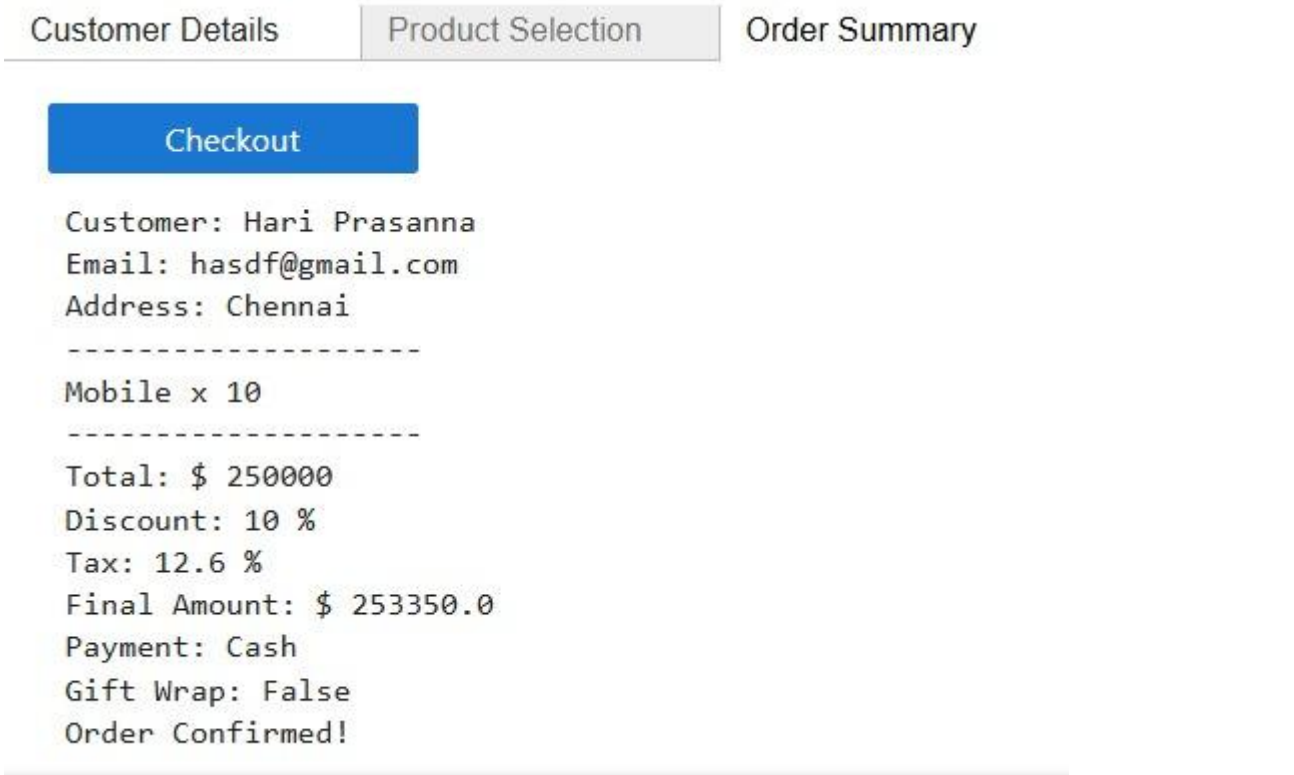
display(tabs)
```

```
%pip install --upgrade jupyterlab_widgets
```

```
%pip install --upgrade ipywidgets
```

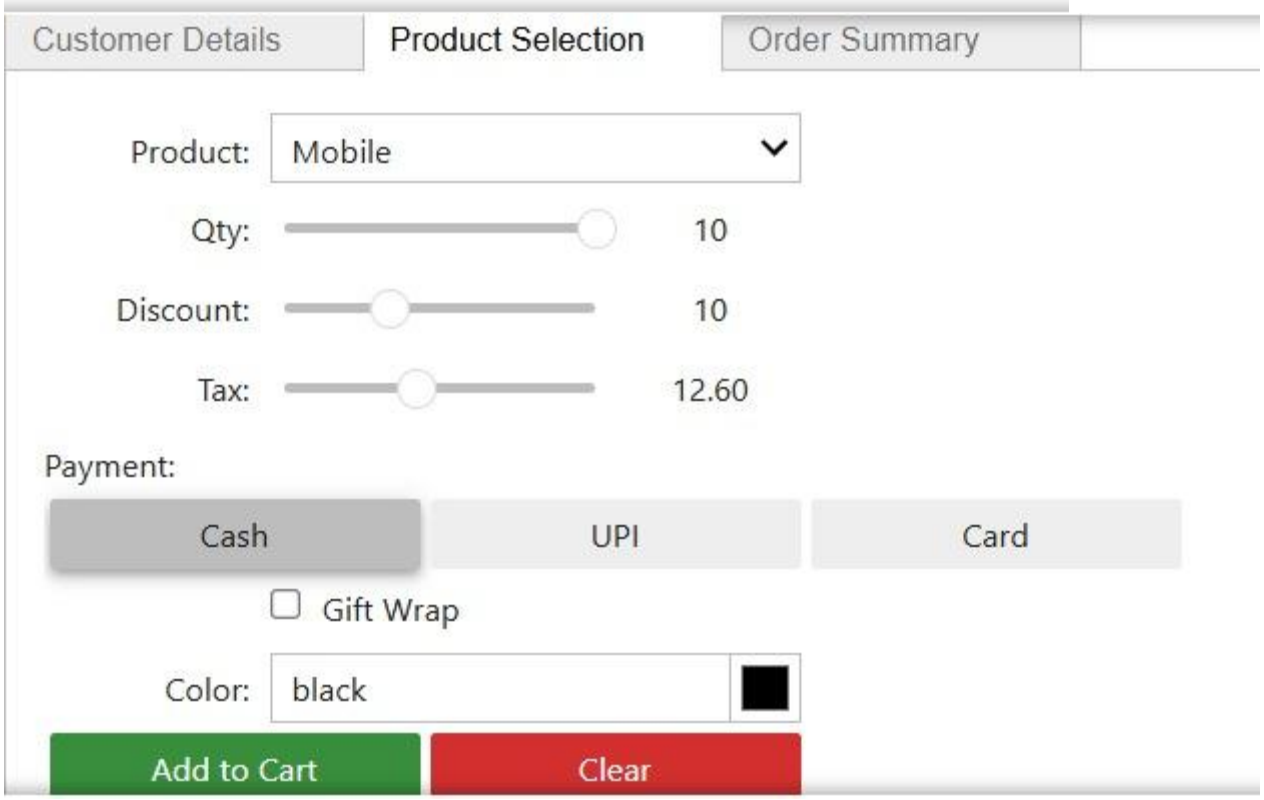
OUTPUT:


Customer Details
Product Selection
Order Summary
Name:
Hari Prasanna
Email:
hasdf@gmail.com
Address:
Chennai


Customer Details
Product Selection
Order Summary
Checkout
Customer: Hari Prasanna
Email: hasdf@gmail.com
Address: Chennai

Mobile x 10

Total: $ 250000
Discount: 10 %
Tax: 12.6 %
Final Amount: $ 253350.0
Payment: Cash
Gift Wrap: False
Order Confirmed!


Customer Details
Product Selection
Order Summary
Product:
Mobile
Qty:
10
Discount:
10
Tax:
12.60
Payment:
Cash
UPI
Card
Gift Wrap
Color:
black
Add to Cart
Clear

RESULT:

Thus, the Python environment was set up with the ipywidgets and jupyterlab_widgets libraries, and an interactive three-tab order form was created in Jupyter Notebook. The Add to Cart, Checkout and Clear buttons are linked to functions that manage the cart and calculate the final bill with discount and tax.

EXP 3

EDA – Data Import and Export

AIM:

To import data into a pandas DataFrame from different sources, namely a CSV file, an Excel file, an SQL (SQLite) database and a web page (web scraping), and to understand how different data formats are handled.

ALGORITHM:

1. Import the libraries pandas, sqlite3, requests, BeautifulSoup and StringIO.
2. Read the Titanic CSV file using read_csv() and display the first five rows.
3. Read the heart statlog Excel file using read_excel() and display the first five rows.
4. Connect to an SQLite database (employee.db) and create a cursor.
5. Create the table employee2 with the columns id, name, department, salary and hire_date.
6. Insert five employee records into the table using executemany() and save them using commit().
7. Read the table into a DataFrame using read_sql() with a SELECT query and display it.
8. Close the database connection.
9. Send a request to the Worldometers web page using requests.get() and check the status code.
10. Parse the page using BeautifulSoup and find all the tables on the page.
11. Convert the first table into a DataFrame using read_html() and display the first five rows.

CODE:

```
import pandas as pd
import sqlite3
import requests
from bs4 import BeautifulSoup
from io import StringIO
```

```
csv_df = pd.read_csv("Titanic-Dataset.csv")
print("CSV Data")
print(csv_df.head())
```

```
excel_df = pd.read_excel("heart stalog dataset.xlsx")
print("Excel Data")
print(excel_df.head())
```

```
conn = sqlite3.connect("employee.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS employee2(
    id INTEGER,
    name TEXT,
```

```
    department TEXT,
    salary REAL,
    hire_date TEXT
)
""")

conn.commit()
```

```
employees = [
    (1,"Alice  Smith","HR",55000,"2018-05-01"),
    (2,"Bob  Johnson","IT",72000,"2019-07-15"),
    (3,"Carol  White","Finance",68000,"2017-09-30"),
    (4,"David  Brown","Marketing",60000,"2020-02-10"),
    (5,"Eva  Green","IT",75000,"2021-04-25")
]

cursor.executemany("""
INSERT INTO employee2
VALUES (?,?,?,?,?)
""", employees)

conn.commit()
```

```
sql_df = pd.read_sql("SELECT * FROM employee2", conn)

print("SQL Data")
print(sql_df.head())
```

```
conn.close()
```

```
url = "https://www.worldometers.info/geography/countries-of-the-world/"
response = requests.get(url)

print("Status Code:", response.status_code)

soup = BeautifulSoup(response.text, "html.parser")
```

```
tables = soup.find_all("table")

print("Total Tables:", len(tables))
```

```
web_df = pd.read_html(StringIO(str(tables[0])))[0]

print("Web Scraped Data")
print(web_df.head())
```

OUTPUT:

Output of: csv_df = pd.read_csv("Titanic-Dataset.csv")

```
CSV Data
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
```

```
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

Output of: excel_df = pd.read_excel("heart stalog dataset.xlsx")

```
Excel Data
   age  sex  chest  resting_blood_pressure  serum_cholestoral  \
0   70    1      4                    130                322
1   67    0      3                    115                564
2   57    1      2                    124                261
3   64    1      4                    128                263
4   74    0      2                    120                269

   fasting_blood_sugar  resting_electrocardiographic_results  \
0                    0                                     2
1                    0                                     2
2                    0                                     0
3                    0                                     0
4                    0                                     2

   maximum_heart_rate_achieved  exercise_induced_angina  oldpeak  slope  \
0                          109                        0      2.4      2
1                          160                        0      1.6      2
2                          141                        0      0.3      1
3                          105                        1      0.2      2
4                          121                        1      0.2      1

   number_of_major_vessels  thal     class
0                        3     3   present
1                        0     7    absent
2                        0     7   present
3                        1     7    absent
4                        1     3    absent
```

Output of: sql_df = pd.read_sql("SELECT * FROM employee2", conn)

```
SQL Data
   id         name department   salary   hire_date
0   1  Alice Smith         HR  55000.0  2018-05-01
1   2  Bob Johnson         IT  72000.0  2019-07-15
2   3  Carol White    Finance  68000.0  2017-09-30
3   4  David Brown  Marketing  60000.0  2020-02-10
4   5    Eva Green         IT  75000.0  2021-04-25
```

Output of: url = "https://www.worldometers.info/geography/countries-of-the-world/"

```
Status Code: 200
```

Output of: tables = soup.find_all("table")

```
Total Tables: 2
```

Output of: web_df = pd.read_html(StringIO(str(tables[0])))[0]

```
Web Scraped Data
   #         Country  Population 2026             Region
0  1           India       1476625576               Asia
1  2           China       1412914089               Asia
2  3   United States        349035494  Northern America
3  4       Indonesia        287886782               Asia
4  5        Pakistan        259299791               Asia
```

RESULT:

Thus, data was imported into pandas DataFrames from a CSV file, an Excel file, an SQLite database table (five employee records) and an HTML table scraped from a web page (status code 200, 2 tables found), and the first rows of each were displayed.

EXP 4

Data Scaling Using Min-Max Normalization, Standardization & Robust Scaling

AIM:

To clean the StudentsPerformance dataset by detecting and filling missing values, removing duplicate rows and making the column names consistent, and to apply Min-Max scaling, Standardization and Robust scaling to the score columns.

ALGORITHM:

1. Import pandas, numpy, matplotlib, seaborn and the MinMaxScaler, StandardScaler and RobustScaler classes from scikit-learn.
2. Load StudentsPerformance.csv into a DataFrame and display the first five rows and the shape.
3. Find the missing values in each column using isnull().sum().
4. Make the column names consistent by removing extra spaces, converting them to lower case and replacing spaces with underscores.
5. Fill the missing values of parental_level_of_education with the mode of the column and verify the result.
6. Remove the duplicate rows using drop_duplicates() and check the new shape.
7. Select the numerical columns (math_score, reading_score and writing_score) and keep a copy of the data.
8. Plot histograms of the scores before normalization.
9. Apply Min-Max scaling to the numerical columns and plot the histograms.
10. Apply Standardization (StandardScaler) to the numerical columns and plot the histograms.
11. Apply Robust scaling (RobustScaler) to the numerical columns and plot the histograms.
12. Compare the distributions before and after scaling.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import MinMaxScaler, StandardScaler,RobustScaler
```

```
df=pd.read_csv('StudentsPerformance.csv')
```

```
df.head()
```

```
df.shape
```

```
df.isnull().sum()
```

```
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
```

```
df.head()
```

```
print(df.isnull().sum())
```

```
df["parental_level_of_education"] =
df["parental_level_of_education"].fillna(df["parental_level_of_education"].mode()[0])
```

```
df.isnull().sum()
```

```
df=df.drop_duplicates()
```

```
df.shape
```

```
numerical_cols = ["math_score", "reading_score", "writing_score"]

print(numerical_cols)
```

```
df_before=df.copy()
```

```
plt.figure(figsize=(15, 4))

for i, col in enumerate(numerical_cols):
    plt.subplot(1, 3, i+1)

    sns.histplot(
        df[col],
        color='red',
        kde=True
    )

    plt.title(f"Before Normalization - {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
minmax_scaler = MinMaxScaler()

df_minmax = df.copy()

df_minmax[numerical_cols] = minmax_scaler.fit_transform(
    df[numerical_cols]
)
```

```
plt.figure(figsize=(15, 4))

for i, col in enumerate(numerical_cols, 1):
    plt.subplot(1, 3, i)

    sns.histplot(
        df_minmax[col],
        kde=True,
        color='green'
    )
```

```
    plt.title(f"Min-Max Normalized - {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
standard_scaler = StandardScaler()

df_standard = df.copy()

df_standard[numerical_cols] = standard_scaler.fit_transform(
    df[numerical_cols]
)
```

```
plt.figure(figsize=(15, 4))

for i, col in enumerate(numerical_cols, 1):
    plt.subplot(1, 3, i)

    sns.histplot(
        df_standard[col],
        kde=True
    )

    plt.title(f"Standardized - {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

```
robust_scaler = RobustScaler()

df_robust = df.copy()

df_robust[numerical_cols] = robust_scaler.fit_transform(
    df[numerical_cols]
)
```

```
plt.figure(figsize=(15, 4))

for i, col in enumerate(numerical_cols, 1):
    plt.subplot(1, 3, i)

    sns.histplot(
        df_robust[col],
        kde=True,
        color='orange'
    )

    plt.title(f"Robust Scaled - {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

OUTPUT:

Output of: df.head()

```
   gender race/ethnicity parental level of education         lunch  \
0  female        group B           bachelor's degree      standard
1  female        group C                some college      standard
2  female        group B             master's degree      standard
3    male        group A          associate's degree  free/reduced
4    male        group C                some college      standard

  test preparation course  math score  reading score  writing score
0                    none          72             72             74
1               completed          69             90             88
2                    none          90             95             93
3                    none          47             57             44
4                    none          76             78             75
```

Output of: df.shape

```
(1005, 8)
```

Output of: df.isnull().sum()

```
gender                       0
race/ethnicity               0
parental level of education  7
lunch                        0
test preparation course      0
math score                   0
reading score                0
writing score                0
dtype: int64
```

Output of: df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

```
   gender race/ethnicity parental_level_of_education         lunch  \
0  female        group B           bachelor's degree      standard
1  female        group C                some college      standard
2  female        group B             master's degree      standard
3    male        group A          associate's degree  free/reduced
4    male        group C                some college      standard

  test_preparation_course  math_score  reading_score  writing_score
0                    none          72             72             74
1               completed          69             90             88
2                    none          90             95             93
3                    none          47             57             44
4                    none          76             78             75
```

Output of: print(df.isnull().sum())

```
gender                       0
race/ethnicity               0
parental_level_of_education  7
lunch                        0
test_preparation_course      0
math score                   0
```

```
reading_score                0
writing_score                0
dtype: int64
```

Output of: df.isnull().sum()

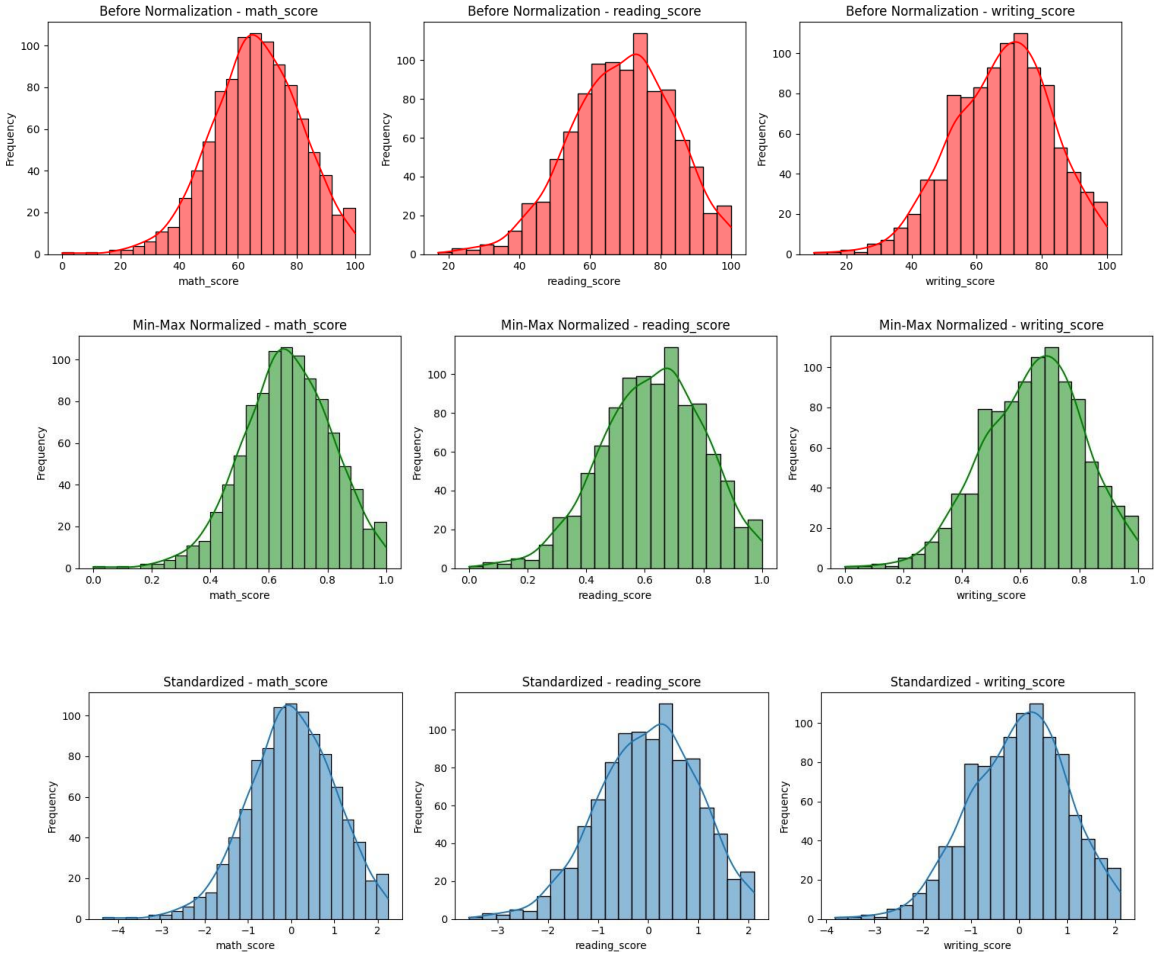
```
gender                       0
race/ethnicity               0
parental_level_of_education  0
lunch                        0
test_preparation_course      0
math_score                   0
reading_score                0
writing_score                0
dtype: int64
```

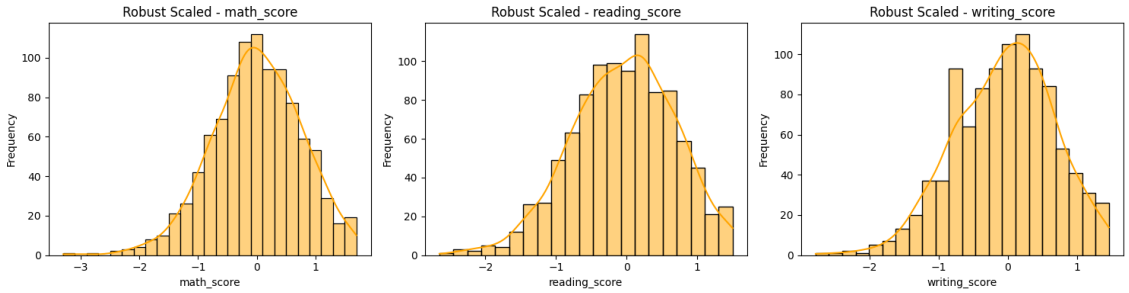
Output of: df.shape

```
(1000, 8)
```

Output of: numerical_cols = ["math_score", "reading_score", "writing_score"]

```
['math_score', 'reading_score', 'writing_score']
```





RESULT:

Thus, the Students Performance dataset was cleaned. The 7 missing values in parental level of education were filled with the mode, 5 duplicate rows were removed (1005 rows reduced to 1000), and Min-Max scaling, Standardization and Robust scaling were applied and compared using histograms.

EXP 5

Exploratory Data Analysis & Data Inspecting

AIM:

To inspect a car dataset using functions such as head(), tail(), shape, columns, dtypes, info() and describe(), to filter the data using conditions, and to find descriptive statistics (mean, median, mode) and plot simple charts.

ALGORITHM:

1. Import pandas, numpy and matplotlib, and load data.csv into a DataFrame.
2. Display the first five rows using head() and the last five rows using tail().
3. Display the number of rows and columns using shape.
4. Display the column names and the data type of each column using columns and dtypes.
5. Display the summary of the dataset using info() and the statistics of numerical columns using describe().
6. Filter the cars that have Highway MPG greater than 30.
7. Filter the BMW cars that have Engine HP greater than 200.
8. Filter the cars that have an Automatic transmission.
9. Select only the Make and MSRP columns.
10. Find the descriptive statistics of Engine HP, MSRP, city mpg and highway MPG.
11. Plot a bar chart of the average MSRP by vehicle size.
12. Plot a histogram and a box plot of Engine HP.
13. Calculate the mean, median and mode of Engine HP and show them as lines on the histogram.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------
# Load Dataset
# ------------------------------------------------------

df = pd.read_csv("data.csv")

# ------------------------------------------------------
# 1. HEAD
# ------------------------------------------------------

print("FIRST 5 ROWS:")
print(df.head())
```

```
# ------------------------------------------------------
# 2. TAIL
# ------------------------------------------------------

print("\nLAST 5 ROWS:")
```

```
print(df.tail())
```

```
# ------------------------------------------------------
# 3. SHAPE
# ------------------------------------------------------

print("\nSHAPE OF DATASET:")
print(df.shape)
```

```
# ------------------------------------------------------
# 4. COLUMNS.TOLIST()
# ------------------------------------------------------

print("\nCOLUMN NAMES:")
print(df.columns)
```

```
# ------------------------------------------------------
# 5. DTYPES
# ------------------------------------------------------

print("\nDATA TYPES:")
print(df.dtypes)
```

```
# ------------------------------------------------------
# 6. INFO
# ------------------------------------------------------

print("\nDATASET INFORMATION:")
df.info()
```

```
# ------------------------------------------------------
# 7. DESCRIBE
# ------------------------------------------------------

print("\nDESCRIPTIVE STATISTICS:")
print(df.describe())
```

```
# ------------------------------------------------------
# 8. Cars with Highway MPG > 30
# ------------------------------------------------------

highway_mpg = df[df['highway MPG'] > 30]

print("\nCARS WITH HIGHWAY MPG > 30:")
print(highway_mpg)
```

```
# ------------------------------------------------------
# 9. BMW Cars with Engine HP > 200
# ------------------------------------------------------

bmw_hp = df[
    (df['Make'] == 'BMW') &
    (df['Engine HP'] > 200)
]

print("\nBMW CARS WITH ENGINE HP > 200:")
print(bmw_hp)
```

```
# ------------------------------------------------------
# 10. Cars with Automatic Transmission
# ------------------------------------------------------

automatic_cars = df[
    df['Transmission Type'] == 'AUTOMATIC'
]

print("\nCARS WITH AUTOMATIC TRANSMISSION:")
print(automatic_cars)
```

```
# ------------------------------------------------------
# 11. Select Make and MSRP columns
# ------------------------------------------------------

selected_columns = df[['Make', 'MSRP']]

print("\nMAKE AND MSRP:")
print(selected_columns)
```

```
# ------------------------------------------------------
# 12. Descriptive Statistics
# ------------------------------------------------------

numeric_columns = [
    'Engine HP',
    'MSRP',
    'city mpg',
    'highway MPG'
]

print("\nDESCRIPTIVE STATISTICS OF SELECTED COLUMNS:")
print(df[numeric_columns].describe())
```

```
# ------------------------------------------------------
# 13. Bar Chart:
#     Average MSRP by Vehicle Size
# ------------------------------------------------------

average_msrp = df.groupby('Vehicle Size')['MSRP'].mean()

plt.figure(figsize=(8, 5))
plt.bar(
    average_msrp.index,
    average_msrp.values
)

plt.xlabel('Vehicle Size')
plt.ylabel('Average MSRP')
plt.title('Average MSRP by Vehicle Size')
plt.xticks(rotation=30)
plt.show()
```

```
# ------------------------------------------------------
# 14. Histogram:
#     Distribution of Engine HP
# ------------------------------------------------------
```

```
hp_data = df['Engine HP'].dropna()

plt.figure(figsize=(8, 5))
plt.hist(
    hp_data,
    bins=20,
    edgecolor='black'
)

plt.xlabel('Engine HP')
plt.ylabel('Frequency')
plt.title('Distribution of Engine HP')
plt.show()
```

```
plt.figure(figsize=(8,6))
plt.boxplot(hp_data, patch_artist=True,
            boxprops=dict(facecolor='lightgreen'))
plt.title('Boxplot of Engine HP')
plt.ylabel('Engine HP')
plt.grid(axis='y', alpha=0.6)
plt.show()
```

```
mean_hp = hp_data.mean()
median_hp = hp_data.median()
mode_hp = hp_data.mode()[0]

plt.figure(figsize=(9, 5))

plt.hist(hp_data, bins=20,color='lightgray', edgecolor='black')

plt.axvline(mean_hp, color='red', linewidth=2.5,
            label=f'Mean = {mean_hp:.2f}')

plt.axvline(median_hp, color='blue', linewidth=2.5,
            label=f'Median = {median_hp:.2f}')

plt.axvline(mode_hp, color='green', linewidth=2.5,
            label=f'Mode = {mode_hp:.2f}')

plt.xlabel('Engine HP')
plt.ylabel('Frequency')
plt.title('Engine HP Distribution with Mean, Median and Mode')
plt.legend()
plt.grid(axis='y', alpha=0.3)

plt.show()
```

OUTPUT:

Output of: import pandas as pd

```
FIRST 5 ROWS:
  Make       Model  Year             Engine Fuel Type  Engine HP  \
0  BMW  1 Series M  2011  premium unleaded (required)      335.0
1  BMW    1 Series  2011  premium unleaded (required)      300.0
2  BMW    1 Series  2011  premium unleaded (required)      300.0
```

```
3  BMW    1 Series  2011  premium unleaded (required)       230.0
4  BMW    1 Series  2011  premium unleaded (required)       230.0

   Engine Cylinders Transmission Type     Driven_Wheels  Number of Doors  \
0               6.0            MANUAL  rear wheel drive              2.0
1               6.0            MANUAL  rear wheel drive              2.0
2               6.0            MANUAL  rear wheel drive              2.0
3               6.0            MANUAL  rear wheel drive              2.0
4               6.0            MANUAL  rear wheel drive              2.0

                        Market Category Vehicle Size Vehicle Style  \
0  Factory Tuner,Luxury,High-Performance       Compact         Coupe
1                     Luxury,Performance       Compact   Convertible
2                Luxury,High-Performance       Compact         Coupe
3                     Luxury,Performance       Compact         Coupe
4                                Luxury       Compact   Convertible

   highway MPG  city mpg  Popularity   MSRP
0           26         19        3916  46135
1           28         19        3916  40650
2           28         20        3916  36350
3           28         18        3916  29450
4           28         18        3916  34500
```

Output of: print("\nLAST 5 ROWS:")

```
LAST 5 ROWS:
          Make    Model  Year                Engine Fuel Type  Engine HP  \
11909    Acura      ZDX  2012     premium unleaded (required)       300.0
11910    Acura      ZDX  2012     premium unleaded (required)       300.0
11911    Acura      ZDX  2012     premium unleaded (required)       300.0
11912    Acura      ZDX  2013  premium unleaded (recommended)       300.0
11913  Lincoln   Zephyr  2006                 regular unleaded       221.0

       Engine Cylinders  Transmission Type      Driven_Wheels  Number of Doors  \
11909               6.0          AUTOMATIC    all wheel drive              4.0
11910               6.0          AUTOMATIC    all wheel drive              4.0
11911               6.0          AUTOMATIC    all wheel drive              4.0
11912               6.0          AUTOMATIC    all wheel drive              4.0
11913               6.0          AUTOMATIC  front wheel drive              4.0

                 Market Category Vehicle Size  Vehicle Style  highway MPG  \
11909  Crossover,Hatchback,Luxury       Midsize   4dr Hatchback           23
11910  Crossover,Hatchback,Luxury       Midsize   4dr Hatchback           23
11911  Crossover,Hatchback,Luxury       Midsize   4dr Hatchback           23
11912  Crossover,Hatchback,Luxury       Midsize   4dr Hatchback           23
11913                      Luxury       Midsize           Sedan           26

       city mpg  Popularity   MSRP
11909        16         204  46120
11910        16         204  56670
11911        16         204  50620
11912        16         204  50920
11913        17          61  28995
```

Output of: print("\nSHAPE OF DATASET:")

```
SHAPE OF DATASET:
(11914, 16)
```

Output of: print("\nCOLUMN NAMES:")

```
COLUMN NAMES:
Index(['Make', 'Model', 'Year', 'Engine Fuel Type', 'Engine HP',
       'Engine Cylinders', 'Transmission Type', 'Driven_Wheels',
       'Number of Doors', 'Market Category', 'Vehicle Size', 'Vehicle Style',
       'highway MPG', 'city mpg', 'Popularity', 'MSRP'],
      dtype='str')
```

Output of: print("\nDATA TYPES:")

```
DATA TYPES:
Make                        str
Model                       str
Year                      int64
Engine  Fuel  Type          str
Engine HP               float64
Engine  Cylinders       float64
Transmission Type           str
Driven_Wheels               str
Number of Doors         float64
Market Category             str
Vehicle Size                str
Vehicle Style               str
highway MPG               int64
city mpg                  int64
Popularity                int64
MSRP                      int64
dtype: object
```

Output of: print("\nDATASET INFORMATION:")

```
DATASET INFORMATION:
<class 'pandas.DataFrame'>
RangeIndex: 11914 entries, 0 to 11913
Data columns (total 16 columns):
 #   Column              Non-Null Count  Dtype
---  -------             --------------  ------
 0   Make                11914 non-null  str
 1   Model               11914 non-null  str
 2   Year                11914 non-null  int64
 3   Engine Fuel Type    11911 non-null  str
 4   Engine HP           11845 non-null  float64
 5   Engine Cylinders    11884 non-null  float64
 6   Transmission Type   11914 non-null  str
 7   Driven_Wheels       11914 non-null  str
 8   Number of Doors     11908 non-null  float64
 9   Market Category     8172 non-null   str
 10  Vehicle Size        11914 non-null  str
 11  Vehicle Style       11914 non-null  str
 12  highway MPG         11914 non-null  int64
 13  city mpg            11914 non-null  int64
 14  Popularity          11914 non-null  int64
```

```
 15  MSRP               11914 non-null  int64
dtypes: float64(3), int64(5), str(8)
memory usage: 2.4 MB
```

Output of: print("\nDESCRIPTIVE STATISTICS:")

```
DESCRIPTIVE STATISTICS:
                Year    Engine HP  Engine Cylinders  Number of Doors  \
count  11914.000000  11845.00000      11884.000000     11908.000000
mean    2010.384338    249.38607          5.628829         3.436093
std        7.579740    109.19187          1.780559         0.881315
min     1990.000000     55.00000          0.000000         2.000000
25%     2007.000000    170.00000          4.000000         2.000000
50%     2015.000000    227.00000          6.000000         4.000000
75%     2016.000000    300.00000          6.000000         4.000000
max     2017.000000   1001.00000         16.000000         4.000000

        highway MPG      city mpg    Popularity          MSRP
count  11914.000000  11914.000000  11914.000000  1.191400e+04
mean      26.637485     19.733255   1554.911197  4.059474e+04
std        8.863001      8.987798   1441.855347  6.010910e+04
min       12.000000      7.000000      2.000000  2.000000e+03
25%       22.000000     16.000000    549.000000  2.100000e+04
50%       26.000000     18.000000   1385.000000  2.999500e+04
75%       30.000000     22.000000   2009.000000  4.223125e+04
max      354.000000    137.000000   5657.000000  2.065902e+06
```

Output of: highway_mpg = df[df['highway MPG'] > 30]

```
CARS WITH HIGHWAY MPG > 30:
         Make       Model  Year                Engine Fuel Type  Engine HP  \
32       FIAT  124 Spider  2017  premium unleaded (recommended)      160.0
33       FIAT  124 Spider  2017  premium unleaded (recommended)      160.0
34       FIAT  124 Spider  2017  premium unleaded (recommended)      160.0
41        BMW     2 Series  2016     premium unleaded (required)      240.0
42        BMW     2 Series  2016     premium unleaded (required)      240.0
...       ...          ...   ...                             ...        ...
11830  Toyota       Yaris  2017               regular unleaded      106.0
11831  Toyota       Yaris  2017               regular unleaded      106.0
11894     BMW          Z4  2014     premium unleaded (required)      240.0
11897     BMW          Z4  2015     premium unleaded (required)      240.0
11901     BMW          Z4  2016     premium unleaded (required)      240.0

       Engine Cylinders Transmission Type      Driven_Wheels  Number of Doors  \
32                  4.0            MANUAL   rear wheel drive              2.0
33                  4.0            MANUAL   rear wheel drive              2.0
34                  4.0            MANUAL   rear wheel drive              2.0
41                  4.0         AUTOMATIC   rear wheel drive              2.0
42                  4.0         AUTOMATIC   rear wheel drive              2.0
...                 ...               ...                ...              ...
11830               4.0         AUTOMATIC  front wheel drive              4.0
11831               4.0            MANUAL  front wheel drive              4.0
11894               4.0            MANUAL   rear wheel drive              2.0
11897               4.0            MANUAL   rear wheel drive              2.0
11901               4.0            MANUAL   rear wheel drive              2.0

         Market Category Vehicle Size  Vehicle Style  highway MPG  city mpg  \
```

```
32            Performance      Compact    Convertible           35        26
33            Performance      Compact    Convertible           35        26
34            Performance      Compact    Convertible           35        26
41     Luxury,Performance      Compact          Coupe           35        23
42                Luxury      Compact    Convertible           34        23
...                  ...          ...            ...          ...       ...
11830           Hatchback      Compact  4dr Hatchback           35        30
11831           Hatchback      Compact  4dr Hatchback           36        30
11894  Luxury,Performance      Compact    Convertible           34        22
11897  Luxury,Performance      Compact    Convertible           34        22
11901  Luxury,Performance      Compact    Convertible           34        22

       Popularity   MSRP
32             819  27495
33             819  24995
34             819  28195
41            3916  32850
42            3916  38650
...            ...    ...
11830         2031  18000
11831         2031  17200
11894         3916  48950
11897         3916  48950
11901         3916  49700

[2905 rows x 16 columns]
```

Output of: bmw_hp = df[

```
BMW CARS WITH ENGINE HP > 200:
      Make        Model  Year             Engine Fuel Type  Engine HP  \
0      BMW  1 Series M  2011  premium unleaded (required)       335.0
1      BMW    1 Series  2011  premium unleaded (required)       300.0
2      BMW    1 Series  2011  premium unleaded (required)       300.0
3      BMW    1 Series  2011  premium unleaded (required)       230.0
4      BMW    1 Series  2011  premium unleaded (required)       230.0
...    ...          ...   ...                           ...        ...
11901  BMW           Z4  2016  premium unleaded (required)       240.0
11902  BMW           Z4  2016  premium unleaded (required)       335.0
11903  BMW           Z8  2001  premium unleaded (required)       394.0
11904  BMW           Z8  2002  premium unleaded (required)       394.0
11905  BMW           Z8  2003  premium unleaded (required)       394.0

       Engine Cylinders Transmission Type      Driven_Wheels  Number of Doors  \
0                   6.0            MANUAL   rear wheel drive              2.0
1                   6.0            MANUAL   rear wheel drive              2.0
2                   6.0            MANUAL   rear wheel drive              2.0
3                   6.0            MANUAL   rear wheel drive              2.0
4                   6.0            MANUAL   rear wheel drive              2.0
...                 ...               ...                ...              ...
11901               4.0            MANUAL   rear wheel drive              2.0
11902               6.0  AUTOMATED_MANUAL   rear wheel drive              2.0
11903               8.0            MANUAL   rear wheel drive              2.0
11904               8.0            MANUAL   rear wheel drive              2.0
11905               8.0            MANUAL   rear wheel drive              2.0

                           Market Category Vehicle Size Vehicle Style  \
0      Factory Tuner,Luxury,High-Performance      Compact         Coupe
```

```
1                        Luxury,Performance       Compact   Convertible
2                   Luxury,High-Performance       Compact         Coupe
3                        Luxury,Performance       Compact         Coupe
4                                   Luxury        Compact   Convertible
...                                    ...           ...           ...
11901                    Luxury,Performance       Compact   Convertible
11902               Luxury,High-Performance       Compact   Convertible
11903         Exotic,Luxury,High-Performance       Compact   Convertible
11904         Exotic,Luxury,High-Performance       Compact   Convertible
11905         Exotic,Luxury,High-Performance       Compact   Convertible

       highway MPG  city mpg  Popularity    MSRP
0               26        19        3916   46135
1               28        19        3916   40650
2               28        20        3916   36350
3               28        18        3916   29450
4               28        18        3916   34500
...            ...       ...         ...     ...
11901           34        22        3916   49700
11902           24        17        3916   66350
11903           19        12        3916  128000
11904           19        12        3916  130000
11905           19        12        3916  131500

[307 rows x 16 columns]
```

Output of: automatic_cars = df[

```
CARS WITH AUTOMATIC TRANSMISSION:
          Make    Model  Year                Engine Fuel Type   Engine HP  \
19        Audi      100  1992                regular unleaded       172.0
23        Audi      100  1993                regular unleaded       172.0
27        Audi      100  1994                regular unleaded       172.0
30        Audi      100  1994                regular unleaded       172.0
31        Audi      100  1994                regular unleaded       172.0
...        ...      ...   ...                             ...         ...
11909    Acura      ZDX  2012     premium unleaded (required)       300.0
11910    Acura      ZDX  2012     premium unleaded (required)       300.0
11911    Acura      ZDX  2012     premium unleaded (required)       300.0
11912    Acura      ZDX  2013  premium unleaded (recommended)       300.0
11913  Lincoln   Zephyr  2006                regular unleaded       221.0

       Engine Cylinders  Transmission Type       Driven_Wheels   Number of Doors  \
19                  6.0          AUTOMATIC    all wheel drive               4.0
23                  6.0          AUTOMATIC    all wheel drive               4.0
27                  6.0          AUTOMATIC  front wheel drive               4.0
30                  6.0          AUTOMATIC  front wheel drive               4.0
31                  6.0          AUTOMATIC    all wheel drive               4.0
...                 ...                ...                ...               ...
11909               6.0          AUTOMATIC    all wheel drive               4.0
11910               6.0          AUTOMATIC    all wheel drive               4.0
11911               6.0          AUTOMATIC    all wheel drive               4.0
11912               6.0          AUTOMATIC    all wheel drive               4.0
11913               6.0          AUTOMATIC  front wheel drive               4.0

                 Market Category Vehicle Size  Vehicle Style  highway MPG  \
19                        Luxury      Midsize          Wagon           20
23                        Luxury      Midsize          Wagon           20
```

```
27                          Luxury      Midsize           Wagon           21
30                          Luxury      Midsize           Sedan           22
31                          Luxury      Midsize           Wagon           21
...                            ...          ...             ...          ...
11909  Crossover,Hatchback,Luxury      Midsize   4dr Hatchback           23
11910  Crossover,Hatchback,Luxury      Midsize   4dr Hatchback           23
11911  Crossover,Hatchback,Luxury      Midsize   4dr Hatchback           23
11912  Crossover,Hatchback,Luxury      Midsize   4dr Hatchback           23
11913                       Luxury      Midsize           Sedan           26

       city mpg  Popularity   MSRP
19           16        3105   2000
23           16        3105   2000
27           16        3105   2000
30           16        3105   2000
31           16        3105   2000
...         ...         ...    ...
11909        16         204  46120
11910        16         204  56670
11911        16         204  50620
11912        16         204  50920
11913        17          61  28995

[8266 rows x 16 columns]
```

Output of: selected_columns = df[['Make', 'MSRP']]

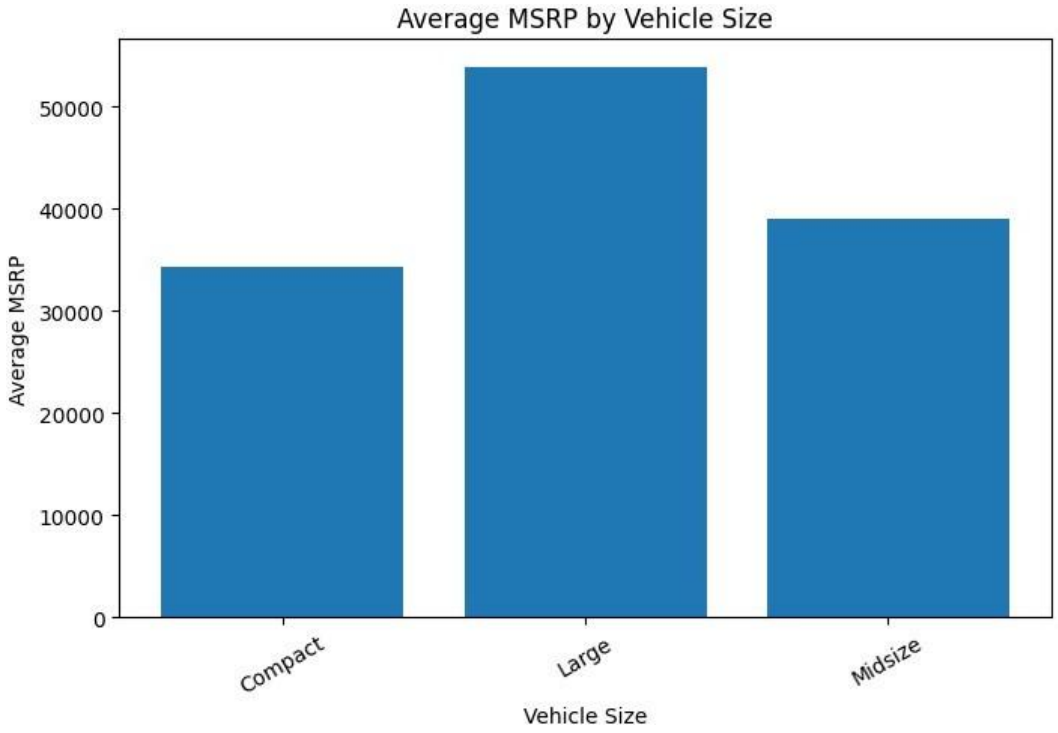
```
MAKE AND MSRP:
          Make   MSRP
0          BMW  46135
1          BMW  40650
2          BMW  36350
3          BMW  29450
4          BMW  34500
...        ...    ...
11909    Acura  46120
11910    Acura  56670
11911    Acura  50620
11912    Acura  50920
11913  Lincoln  28995

[11914 rows x 2 columns]
```

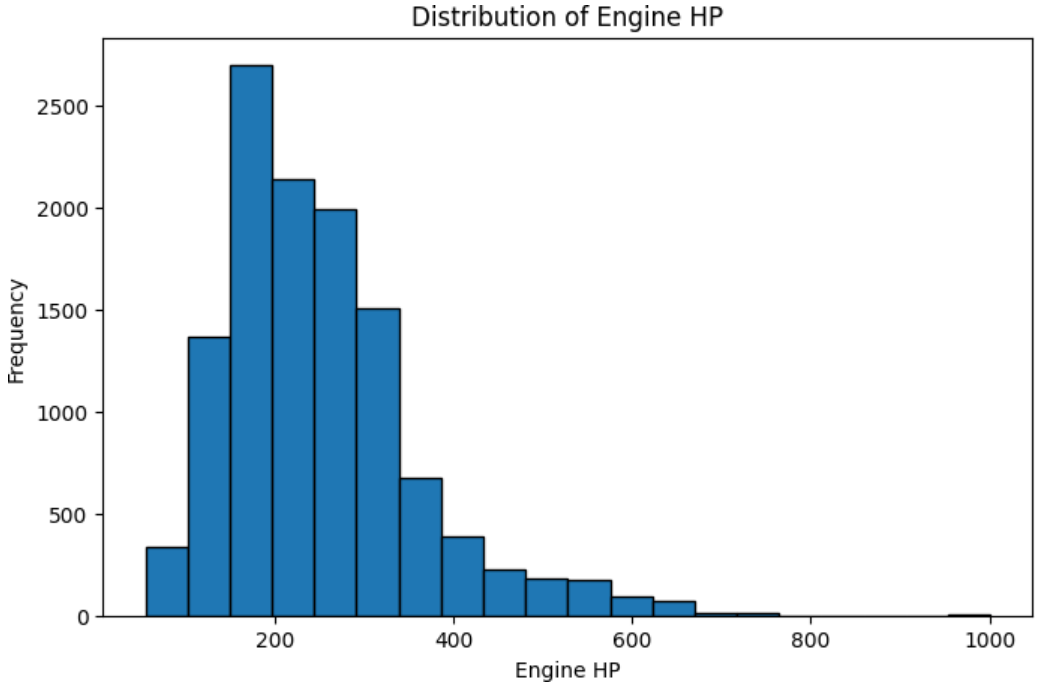
Output of: numeric_columns = [

```
DESCRIPTIVE STATISTICS OF SELECTED COLUMNS:
         Engine HP           MSRP       city mpg   highway MPG
count  11845.00000   1.191400e+04  11914.000000  11914.000000
mean     249.38607   4.059474e+04     19.733255     26.637485
std      109.19187   6.010910e+04      8.987798      8.863001
min       55.00000   2.000000e+03      7.000000     12.000000
25%      170.00000   2.100000e+04     16.000000     22.000000
50%      227.00000   2.999500e+04     18.000000     26.000000
75%      300.00000   4.223125e+04     22.000000     30.000000
max     1001.00000   2.065902e+06    137.000000    354.000000
```

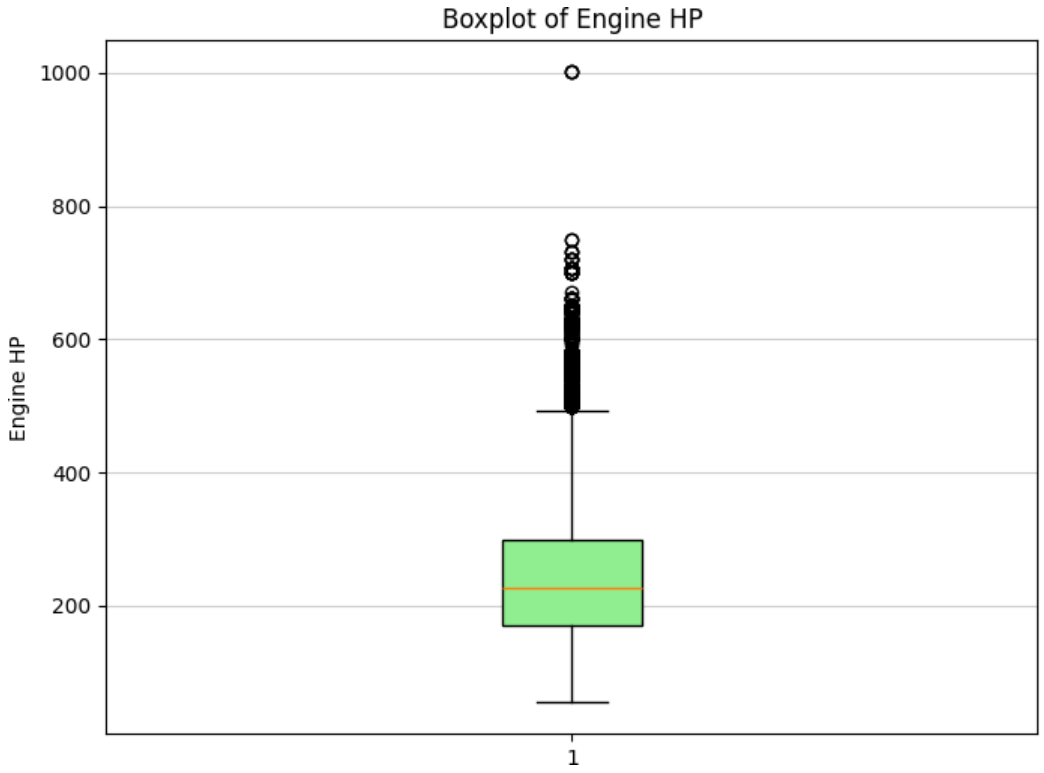
Output of: average_msrp = df.groupby('Vehicle Size')['MSRP'].mean()



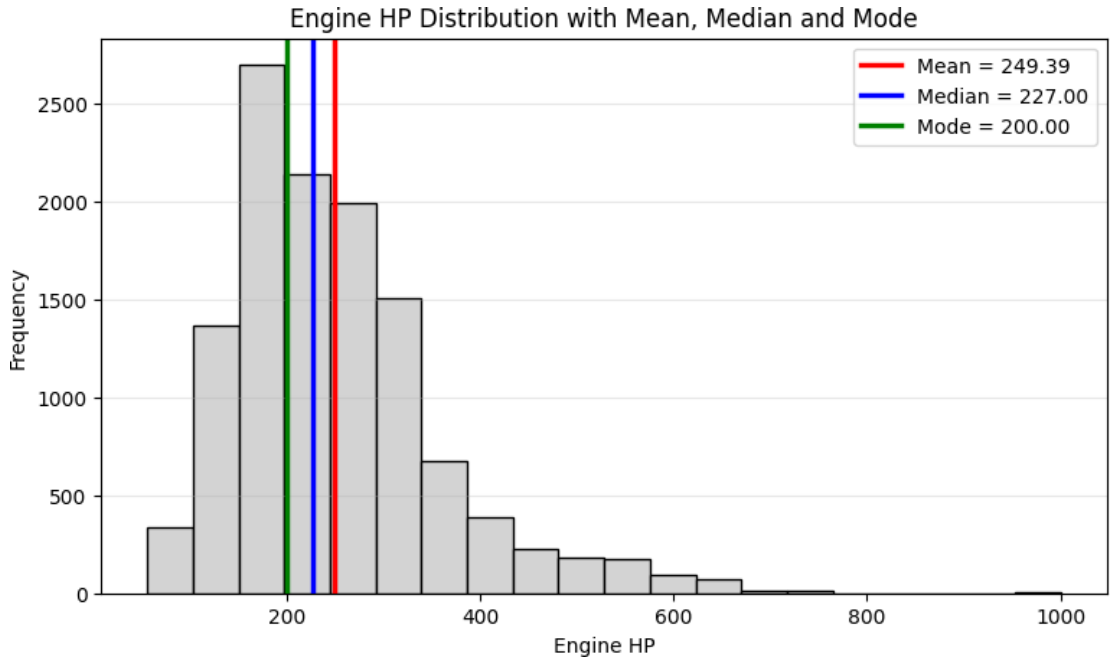
Output of: hp_data = df['Engine HP'].dropna()



Output of: plt.figure(figsize=(8,6))



Output of: mean_hp = hp_data.mean()



RESULT:

Thus, the car dataset (11914 rows and 16 columns) was inspected and analysed. The data was filtered using conditions, descriptive statistics were calculated (for example, the mean Engine HP is 249.39 and the median is 227), and the bar chart, histogram, box plot and mean-median-mode plot were drawn.

EXP 6

Exploratory Data Visualization

AIM:

To understand the patterns in the student performance dataset by drawing a line chart, a bar chart and a histogram, and to study how study time affects the marks and how the number of absences is distributed.

ALGORITHM:

1. Import pandas, matplotlib and numpy.
2. Load the student performance file (stu_per.csv) into a DataFrame and display the first five rows.
3. Group the data by studytime and calculate the average of the marks G1, G2 and G3.
4. Draw a line chart of the average G1, G2 and G3 marks for each study time.
5. Draw a bar chart of the average final mark (G3) for each study time.
6. Draw a histogram of the number of absences of the students.
7. Observe the charts and write the result.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np



df = pd.read_csv("stu_per.csv")

print(df.head())
```

```
# Group by study time and calculate average scores
grouped_data = df.groupby("studytime")[["G1", "G2", "G3"]].mean()

print(grouped_data)

plt.figure(figsize=(8, 5))

plt.plot(
    grouped_data.index,
    grouped_data["G1"],
    marker="o",
    label="G1"
)

plt.plot(
    grouped_data.index,
    grouped_data["G2"],
    marker="s",
    label="G2"
)
```

```
plt.plot(
    grouped_data.index,
    grouped_data["G3"],
    marker="^",
    label="G3"
)

plt.title("Study Time vs Student Performance")
plt.xlabel("Study Time")
plt.ylabel("Average Score")
plt.legend()
plt.grid(True)

plt.show()
```

```
study_score = df.groupby("studytime")["G3"].mean()

plt.figure(figsize=(8, 5))

plt.bar(
    study_score.index,
    study_score.values
)

plt.title("Average Final Score by Study Time")
plt.xlabel("Study Time")
plt.ylabel("Average G3 Score")

plt.show()
```

```
plt.figure(figsize=(8, 5))

plt.hist(
    df["absences"],
    bins=10,
    edgecolor="black"
)

plt.title("Distribution of Student Absences")
plt.xlabel("Number of Absences")
plt.ylabel("Number of Students")

plt.show()
```

OUTPUT:

Output of: import pandas as pd

```
  school sex  age address famsize Pstatus  Medu  Fedu      Mjob      Fjob  ...  \
0     GP   F   18       U     GT3       A     4     4   at_home   teacher  ...
1     GP   F   17       U     GT3       T     1     1   at_home     other  ...
2     GP   F   15       U     LE3       T     1     1   at_home     other  ...
3     GP   F   15       U     GT3       T     4     2    health  services  ...
4     GP   F   16       U     GT3       T     3     3     other     other  ...

   famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0       4        3      4     1     1      3        4   0  11  11
```

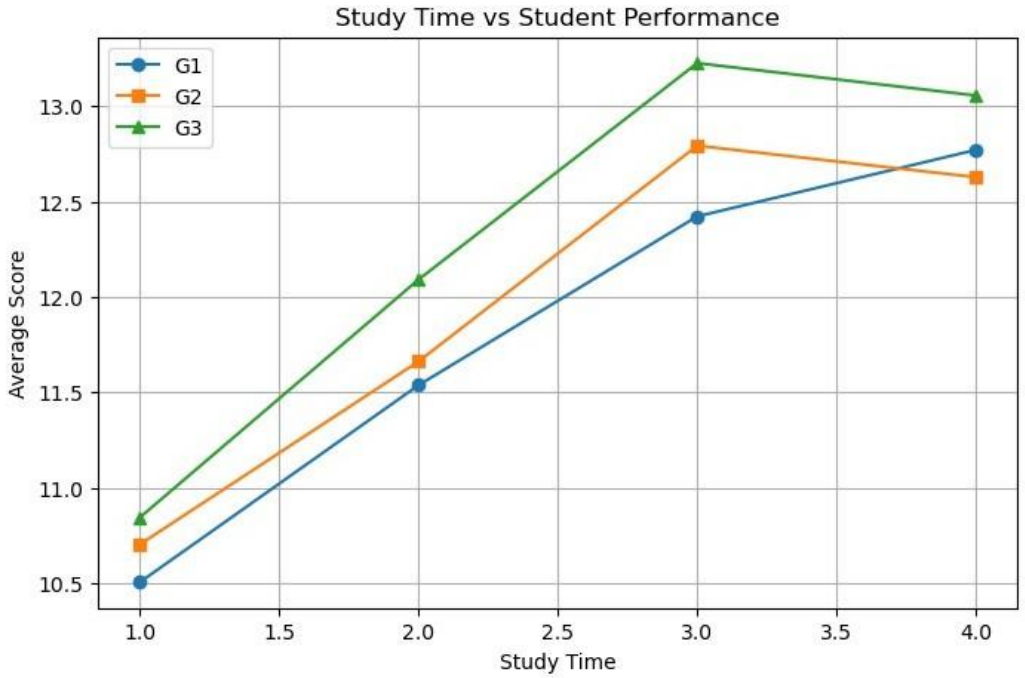
```
1       5         3       3       1       1       3         2    9   11   11
2       4         3       2       2       3       3         6   12   13   12
3       3         2       2       1       1       5         0   14   14   14
4       4         3       2       1       2       5         0   11   13   13

[5 rows x 33 columns]
```

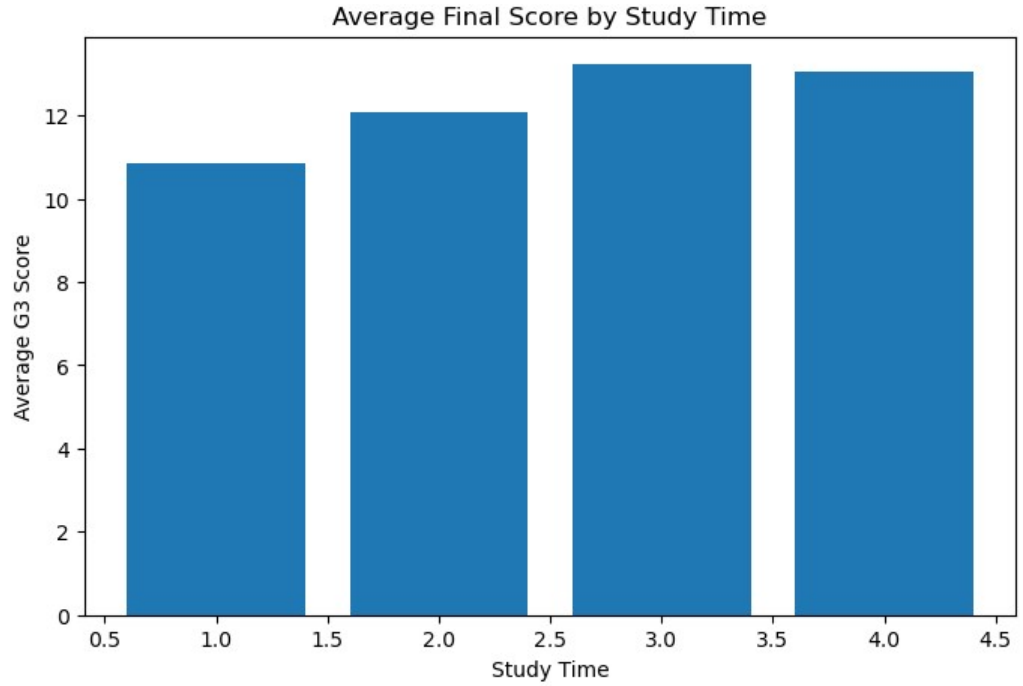
Output of: grouped_data = df.groupby("studytime")[["G1", "G2", "G3"]].mean()

```
                 G1         G2         G3
studytime
1          10.504717  10.702830  10.844340
2          11.537705  11.662295  12.091803
3          12.422680  12.793814  13.226804
4          12.771429  12.628571  13.057143
```

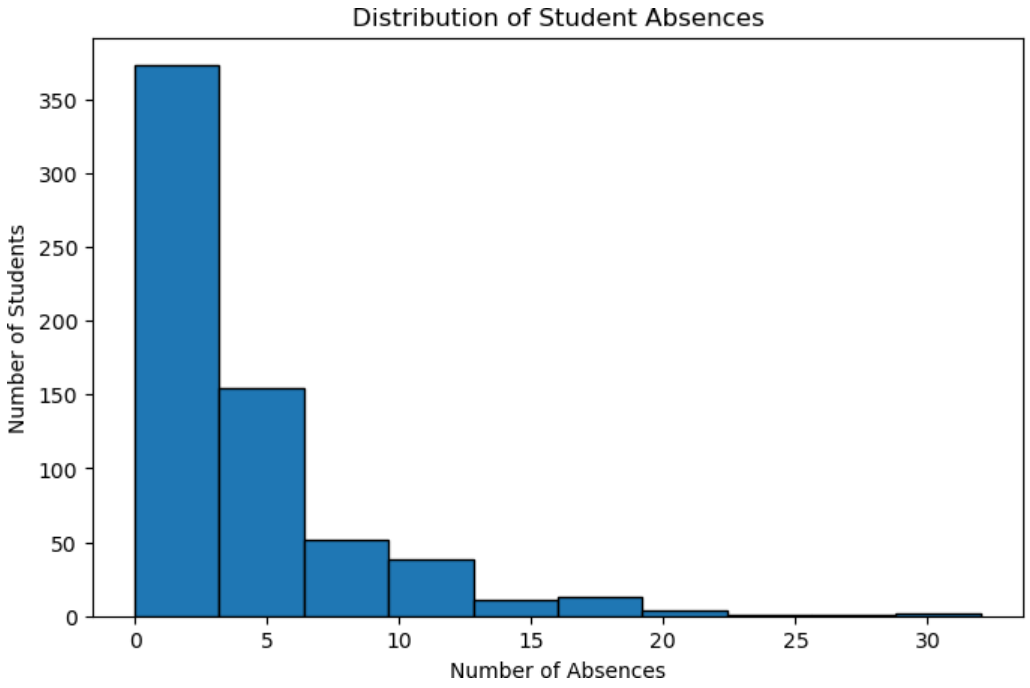
Output of: grouped_data = df.groupby("studytime")[["G1", "G2", "G3"]].mean()



Output of: study_score = df.groupby("studytime")["G3"].mean()



Output of: plt.figure(figsize=(8, 5))



RESULT:

Thus, the student performance data was visualized using a line chart, a bar chart and a histogram. The average final mark (G3) increased from 10.84 for study time 1 to 13.23 for study time 3, showing that students who study more tend to score higher marks.

EXP 7

Data Visualization Using Power BI

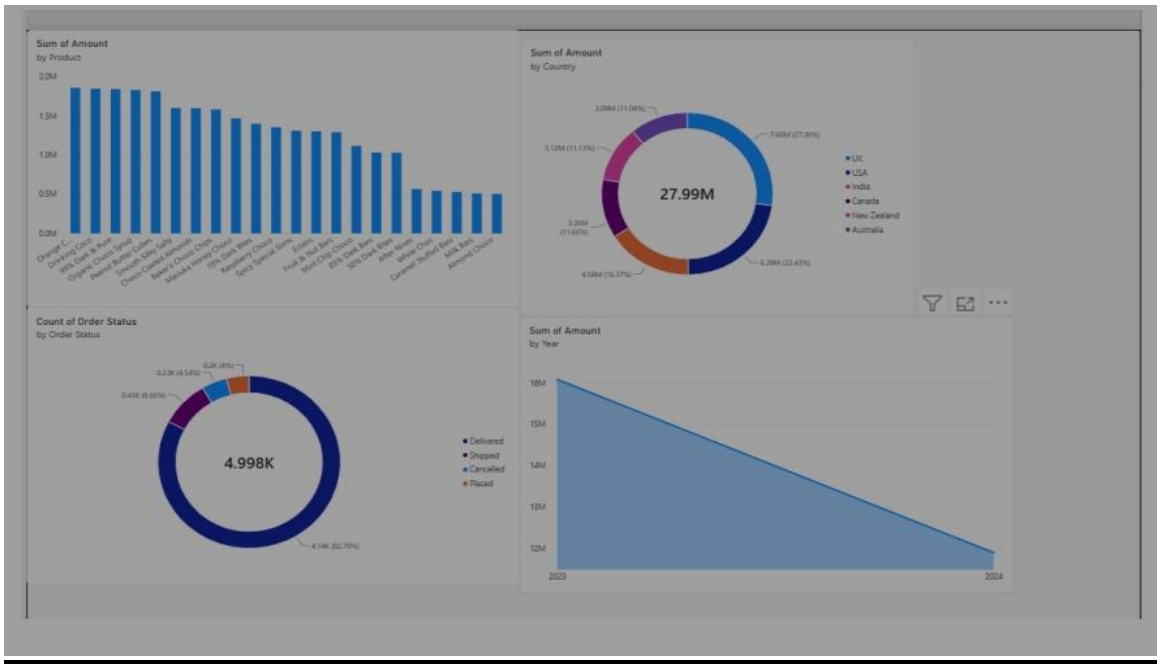
AIM:

To learn the Power BI Desktop workflow by connecting to Excel and CSV data, cleaning the data in Power Query, creating basic visualizations (bar, line and pie charts, cards and tables) and arranging them as an interactive dashboard.

ALGORITHM:

1. Open Power BI Desktop and identify the Fields, Visualizations and Filters panes and the Report, Data and Model views.
2. Click Home → Get data → Excel, select the sales file, choose the required sheet in the Navigator and click Load.
3. Click Home → Get data → Text/CSV, select the targets file, preview it and click Load.
4. Click Transform data to open the Power Query Editor.
5. Set the correct data types, remove extra spaces and duplicate rows, and add a Total Sales column (Quantity × Unit Price).
6. Click Close & Apply to load the cleaned data into the report.
7. Create a bar chart of sales by category and a line chart of the monthly sales trend.
8. Create a pie or donut chart of sales by region and card visuals for total sales and total quantity.
9. Create a table of the top 10 products by sales using a Top N filter.
10. Add slicers (for example Region or Date) and arrange all visuals neatly on one report page as a dashboard.
11. Add titles and consistent formatting, then save the report as a .pbix file.

OUTPUT:



RESULT:

Thus, the Power BI Desktop workflow was studied. Data was imported from Excel and CSV sources, cleaned using Power Query, and visualized using bar, line and pie charts, cards and tables, and an interactive dashboard was prepared.